Sea Lyon Media Manager — macOS Apple Silicon Implementation Plan

Target: 100% Apple Silicon (arm64) only. macOS 11+ on M1/M2/M3/M4. **No Intel Mac support, not even via Rosetta** — the shipped Mach-O binaries contain zero `x86_64` slices, so the app will not launch on Intel hardware. The deliverable is a single self-contained `.app` inside a signed/notarized `.dmg`.

Source branch: `LMM-DEV` → continue work on `claude/macos-apple-silicon-port-SXySX` (single codebase).

Differs from the audit report on three points (per project owner):

1. **Disc functionality is included** on macOS — full detection, playback, and ripping.
2. **Disc and Rip tabs are hidden only when no drive is connected** (live detection, not hard-disable).
3. **The DMG is one inclusive package** — libVLC, ffmpeg, fpcalc, and libdiscid all live inside the `.app` bundle. No Homebrew prerequisite, no separate VLC.app dependency.

Architecture rules (binding for every phase)

- **Build host must be Apple Silicon** (`macos-14` runner in CI, M1+ on developer machines). No cross-compilation from Intel.
- **Python must be arm64-native**, not running under Rosetta. CI uses `actions/setup-python@v5` with `architecture: arm64`; locally use `python -c "import platform; assert platform.machine() == 'arm64'"` before any work.
- **PyInstaller is invoked with** `--target-arch arm64` so it refuses to produce a universal2 binary.
- **Universal2 wheels (PyObjC, PySide6, etc.) are accepted at install time but thinned to arm64 in the bundle.** A mandatory post-build step uses `lipo -thin arm64` on every Mach-O file inside the `.app`, then `lipo -archs` verifies the final result equals exactly `arm64` — anything else fails the build.
- `LSMinimumSystemVersion = 11.0` in Info.plist (macOS Big Sur, the first arm64-capable macOS).
- **DMG file name carries the architecture**: `SeaLyonMediaManager-<ver>-arm64.dmg`. No `-universal` or `-x86_64` variants are ever produced.

---

0. Pre-flight (before Phase 1)

0.1 Branch hygiene

- Continue on `claude/macos-apple-silicon-port-SXySX` for development.
- Land each phase as its own PR into `LMM-DEV`. The Windows build must stay green throughout.

0.2 One-time accounts and assets

- **Apple Developer Program** membership ($99/year). Without this no notarization, no DMG that opens cleanly for end users.
- Generate a **Developer ID Application** certificate and a **Developer ID Installer** certificate (we only need Application unless we ship a `.pkg`).
- Create an **app-specific password** for the Apple ID used by `notarytool`.
- GitHub Actions secrets to add:
  - `APPLE_DEVELOPER_ID_CERT_P12` — base64-encoded `.p12` of the Application cert + private key.
  - `APPLE_DEVELOPER_ID_CERT_PASSWORD` — password protecting the `.p12`.
  - `APPLE_ID` — Apple ID email.
  - `APPLE_TEAM_ID` — 10-character team identifier from developer.apple.com.
  - `APPLE_APP_PASSWORD` — app-specific password.

0.3 Local dev environment (each engineer)

- macOS 14+ on an M1/M2/M3/M4 Mac. **Intel Macs cannot build or run this port** — there is no cross-compile path.
- Xcode Command Line Tools: `xcode-select --install`.
- Python 3.11 arm64-native: either `pyenv install 3.11` on an arm64 shell, or the python.org universal2 installer launched without Rosetta. Verify with `python -c "import platform; assert platform.machine()=='arm64'"`.
- `brew install create-dmg dylibbundler` (build-time tools; not bundled into the app).
- When running PyInstaller manually for iteration: `python -m PyInstaller --noconfirm --target-arch arm64 build/lyon.spec`. The `--target-arch arm64` flag is mandatory — without it PyInstaller produces a universal2 binary on a universal Python.

0.4 Test matrix to define up-front

Hardware	macOS	Disc drive	Required outcome
M1 Mac mini	11.0 (Big Sur)	Apple USB SuperDrive	Full disc rip + playback
M2 MacBook Air	14.0 (Sonoma)	None	Disc/Rip tabs hidden until drive plugged in
M3 Pro	15.0 (Sequoia)	Pioneer external USB	Full disc rip + playback
M1 (any)	11.0	Plug in / unplug mid-session	Tabs appear/disappear within 1 second

These four scenarios are the acceptance bar for the port.

0.5 Total scope and rough effort

Phase	Days	Critical path?
1. Foundations (paths, font, window mode, deps)	1	yes
2. Bundled runtime discovery (libVLC, ffmpeg, fpcalc, libdiscid)	2	yes
3. macOS disc detection (IOKit + DiskArbitration hot-plug)	3	yes
4. macOS disc rip + playback wiring	2	yes
5. UI parity (QSS, shortcuts, tab gating)	2	yes
6. Build pipeline (.icns, .app, codesign, notarize, DMG)	3	yes
7. CI workflow (macos-14 runner)	1	yes
8. Updater (per-OS appcast)	1	parallel to 6
9. QA across hardware + release	2	yes
Total	**17 days**	

Phase 1 — Foundations

Goal: Application launches on Apple Silicon, writes to the correct directories, looks native at first paint, and has the right Python deps installed.

1.1 macOS application-data directory

File: `lyon/core/settings.py:62-69`

Replace the current two-branch `app_data_dir()`:

```
def app_data_dir() -> Path:
    if sys.platform == "win32":
        base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
    elif sys.platform == "darwin":
        base = str(Path.home() / "Library" / "Application Support")
    else:
        base = os.environ.get("XDG_CONFIG_HOME") or str(Path.home() / ".config")
    p = Path(base) / "LyonMusicManager"
    p.mkdir(parents=True, exist_ok=True)
    return p
```

Add a one-shot migration helper called from `lyon/app.py` before `Settings.load()`:

```
def _migrate_macos_app_data() -> None:
    if sys.platform != "darwin":
        return
    legacy = Path.home() / ".config" / "LyonMusicManager"
    new = Path.home() / "Library" / "Application Support" / "LyonMusicManager"
    if legacy.exists() and not new.exists():
        new.parent.mkdir(parents=True, exist_ok=True)
        legacy.rename(new)
        LOG.info("Migrated app data from %s to %s", legacy, new)
```

Acceptance: fresh launch on a clean Mac creates `~/Library/Application Support/LyonMusicManager/settings.json` and `…/logs/sea-lyon.log`. Existing `~/.config/LyonMusicManager` is migrated.

1.2 Application font

File: `lyon/app.py:113-118`

```
if sys.platform == "win32":
    app.setFont(QFont("Segoe UI", 9))
elif sys.platform == "darwin":
    app.setFont(QFont(".AppleSystemUIFont", 13))
else:
    app.setFont(QFont("Ubuntu", 10))
```

`.AppleSystemUIFont` resolves to whatever the system uses (SF Pro on 11+). 13pt is the standard sidebar/control size; the QSS will no longer override it after Phase 5.

1.3 Window mode

File: `lyon/app.py:124-127`

```
win = MainWindow()
win.setWindowIcon(icon)
if sys.platform == "darwin":
    win.show()
else:
    win.showMaximized()
finish_startup_splash(app, win)
```

On macOS the window appears at the size set in `main_window.py:189` (`resize(1100, 720)`). A follow-up should persist last geometry, but that's polish, not blocker.

1.4 macOS-specific Python dependencies

File: `requirements.in`

Replace the `discid` line with two markers (keep Windows-only pin, add darwin pin):

```
PySide6-Essentials==6.11.1
mutagen==1.47.0
musicbrainzngs==0.7.1
discid==1.3.0; sys_platform == "win32"
discid==1.3.0; sys_platform == "darwin"
requests==2.34.2
defusedxml==0.7.1
python-vlc==3.0.21203
yt-dlp==2026.3.17
watchdog==6.0.0
pyacoustid==1.3.1
pyobjc-framework-Cocoa==10.3.1;     sys_platform == "darwin"
pyobjc-framework-IOKit==10.3.1;     sys_platform == "darwin"
pyobjc-framework-DiskArbitration==10.3.1; sys_platform == "darwin"
```

Re-pin compile with `pip-compile --output-file requirements.txt requirements.in` **on an arm64 host**. PyObjC and PySide6 wheels arrive as `*-macosx_11_0_universal2.whl` (they contain both arm64 and x86_64 slices); installing them is fine — the x86_64 halves are stripped from the final bundle in Phase 6.3. Add a build-time assert to `requirements-build.txt` resolution: `python -c "import platform; assert platform.machine() == 'arm64', 'Apple Silicon required'"`.

1.5 Acceptance for Phase 1

- `python main.py` launches on an Apple Silicon Mac and shows the main window at 1100×720.
- Window title reads "Sea Lyon Media Manager".
- Settings file lands in `~/Library/Application Support/LyonMusicManager/settings.json`.
- The font in dialogs is San Francisco, not Tahoma fallback.

Phase 2 — Bundled Runtime Discovery

Goal: When the .app launches, it finds the **bundled** libVLC, ffmpeg, fpcalc, and libdiscid inside its own `Contents/` tree. The user does not need Homebrew, VLC.app, or any other install.

2.1 Bundle layout decision

Inside the .app:

```
Sea Lyon Media Manager.app/
├── Contents/
│   ├── Info.plist
│   ├── MacOS/
│   │   ├── LyonMusicManager          # PyInstaller entry binary
│   │   └── bin/
│   │       ├── ffmpeg                # arm64 static binary
│   │       └── fpcalc                # arm64 Chromaprint binary
│   ├── Frameworks/
│   │   ├── libvlc.dylib              # libVLC core
│   │   ├── libvlccore.dylib
│   │   ├── libdiscid.0.dylib         # libdiscid for AccurateRip
│   │   └── plugins/                  # libVLC plugins directory
│   │       ├── access/
│   │       ├── audio_output/
│   │       └── ...
│   └── Resources/
│       ├── lyon-app-icon.icns
│       └── (Qt resources injected by PyInstaller)
```

Why this layout:

- Binaries the app spawns via `subprocess` (ffmpeg, fpcalc) go in `Contents/MacOS/bin/`. Apple permits executables there; `bin/` subdir is fine.
- Dylibs the app loads via `ctypes` / `dlopen` (libvlc, libdiscid) go in `Contents/Frameworks/`. This is the directory Apple's `dyld` searches via `@executable_path/../Frameworks` rpath, and the directory `codesign --deep` traverses.
- libVLC's `plugins/` folder must sit beside `libvlc.dylib` and be referenced via `VLC_PLUGIN_PATH`.

2.2 Runtime discovery code

File: `lyon/core/settings.py:72-76`

Extend `bundled_bin_dir()` to know about `Contents/MacOS/bin` when frozen on macOS:

```
def bundled_bin_dir() -> Path:
    if getattr(sys, "frozen", False):
        # PyInstaller sets sys._MEIPASS; on macOS .app bundles this points at
        # Contents/Frameworks (onefile) or Contents/MacOS/_internal (onedir).
        # We always place ffmpeg/fpcalc under MacOS/bin in the BUNDLE step.
        if sys.platform == "darwin":
            return Path(sys.executable).resolve().parent / "bin"
        return Path(sys._MEIPASS) / "bin"  # type: ignore[attr-defined]
    return Path(__file__).resolve().parent.parent.parent / "bin"


def bundled_frameworks_dir() -> Path | None:
    """Path to Contents/Frameworks inside a .app bundle, or None when not frozen on macOS."""
    if sys.platform != "darwin" or not getattr(sys, "frozen", False):
        return None
    # sys.executable: …/Sea Lyon Media Manager.app/Contents/MacOS/LyonMusicManager
    return Path(sys.executable).resolve().parent.parent / "Frameworks"
```

2.3 libVLC discovery — darwin branch

File: `lyon/core/playback_backend.py:44-68`

Replace `_configure_vlc_runtime_path()`:

```
def _configure_vlc_runtime_path() -> None:
    """Make libVLC importable regardless of platform.

    Priority on darwin:
    1. Bundled Contents/Frameworks/libvlc.dylib (production .app)
    2. /Applications/VLC.app/Contents/MacOS/lib (dev fallback)
    3. python-vlc's own search (lets users override via env)
    """
    if sys.platform == "win32":
        _configure_vlc_runtime_path_windows()
    elif sys.platform == "darwin":
        _configure_vlc_runtime_path_darwin()


def _configure_vlc_runtime_path_darwin() -> None:
    candidates: list[Path] = []
    fw = bundled_frameworks_dir()
    if fw and (fw / "libvlc.dylib").exists():
        candidates.append(fw)
    candidates.extend([
        Path("/Applications/VLC.app/Contents/MacOS/lib"),
    ])
    for path in candidates:
        if (path / "libvlc.dylib").exists():
            os.environ.setdefault("DYLD_FALLBACK_LIBRARY_PATH", str(path))
            plugins = path / "plugins" if (path / "plugins").exists() else path / "vlc" / "plug
ins"
```

```
            if plugins.exists():
                os.environ.setdefault("VLC_PLUGIN_PATH", str(plugins))
            _CONFIGURED_VLC_DIRS.add(path)
            return
```

Move the existing Windows logic into `_configure_vlc_runtime_path_windows()` (mechanical extraction, keep behavior identical).

2.4 ffmpeg and fpcalc — already cross-platform

`lyon/core/ffmpeg.py:10-18` and `lyon/core/fingerprint.py:37-50,73` already check `bundled_bin_dir()` first then PATH. Once we drop static arm64 binaries into `Contents/MacOS/bin/ffmpeg` and `…/fpcalc`, no code change is required.

2.5 libdiscid — explicit dylib load on macOS

The Python `discid` package locates libdiscid by `ctypes.util.find_library` which searches `DYLD_LIBRARY_PATH` and the system. Inside a sandboxed .app, that often misses bundled dylibs. Add an explicit pre-load in `lyon/core/cd_detect.py`:

```
def _preload_libdiscid_darwin() -> None:
    if sys.platform != "darwin":
        return
    fw = bundled_frameworks_dir()
    if fw is None:
        return
    candidate = fw / "libdiscid.0.dylib"
    if candidate.exists():
        try:
            ctypes.CDLL(str(candidate), mode=ctypes.RTLD_GLOBAL)
        except OSError as exc:
            LOG.warning("Could not preload bundled libdiscid: %s", exc)


_preload_libdiscid_darwin()
```

Run this before the `import discid` happens elsewhere; placing it at module top of `cd_detect.py` (before line 11) is sufficient because every disc code path goes through `cd_detect`.

2.6 Fetching the binaries — arm64-only sources

Every external binary we bundle must be **arm64-only** (or a universal binary that gets thinned in Phase 6.3). We download all of these in CI (Phase 7); this section enumerates **what** to fetch and **how** to verify arch:

Binary	Source (arm64)	Verify
ffmpeg	`https://evermeet.cx/ffmpeg/ffmpeg-7.1.1.zip` — pick the arm64 release explicitly, not the Intel build. evermeet.cx publishes separate Intel and Apple Silicon archives.	SHA-256 from `evermeet.cx/ffmpeg/info/ffmpeg/release` JSON
fpcalc	`https://github.com/acoustid/chromaprint/releases/download/v1.5.1/chromaprint-fpcalc-1.5.1-macos-arm64.tar.gz`	SHA-256 from GitHub release
libvlc + plugins	`https://download.videolan.org/pub/videolan/vlc/3.0.21/macosx/vlc-3.0.21-arm64.dmg` — VLC publishes a dedicated arm64 build since 3.0.18. **Do not use** `vlc-3.0.21-intel64.dmg`.	SHA-256 from `vlc-3.0.21-arm64.dmg.sha256`
libdiscid	Build from source on the macos-14 arm64 runner: `./configure --host=arm64-apple-darwin && make`. Cache the resulting `libdiscid.0.dylib` keyed by version.	SHA-256 of upstream tarball `libdiscid-0.6.4.tar.gz`

Mandatory arch verification after each download — fail the build if any slice is not arm64:

```
verify_arm64_only() {
    local f="$1"
    local archs
    archs=$(lipo -archs "$f" 2>/dev/null || echo "unknown")
    if [ "$archs" != "arm64" ]; then
        echo "FAIL: $f arch is '$archs' (expected exactly 'arm64')"
        exit 1
    fi
}

verify_arm64_only vendor-mac/ffmpeg
verify_arm64_only vendor-mac/fpcalc
verify_arm64_only vendor-mac/libdiscid.0.dylib
# libvlc and plugins: verify after extraction from the dmg
for f in vendor-mac/vlc/lib/libvlc.dylib vendor-mac/vlc/lib/libvlccore.dylib vendor-mac/vlc/plu
gins/**/*.dylib; do
    verify_arm64_only "$f"
done
```

The VLC `arm64.dmg` ships arm64-only dylibs by design — but verifying guarantees the upstream didn't silently switch to universal2.

2.7 Acceptance for Phase 2

- `from lyon.core.playback_backend import create_playback_backend` returns a `VlcPlaybackBackend` (not `UnavailablePlaybackBackend`) on a clean Mac with no VLC.app installed, when run inside the packaged .app.
- `find_ffmpeg_binary()` returns the bundled path.

- `lyon.core.fingerprint.is_available()` returns `True`.
- Playing an MP3 from the library produces audio.

Phase 3 — macOS Optical Disc Detection

Goal: Detect optical drives and audio CDs at runtime via IOKit / DiskArbitration. The UI's "Disc" and "Rip" tabs appear when a drive is present, disappear when removed.

3.1 New module: `lyon/core/disc_macos.py`

Create a dedicated module so the Win32 ctypes path in `cd_detect.py` stays isolated. Skeleton:

```
"""macOS optical-drive detection via IOKit + DiskArbitration.

All symbols are imported lazily inside functions so this module remains
import-safe on non-darwin platforms (it shouldn't actually be imported there,
but defensive guards stop accidental test runs from crashing).
"""
from __future__ import annotations

import logging
import subprocess
import sys
from dataclasses import dataclass
from pathlib import Path
from typing import Callable

LOG = logging.getLogger(__name__)


@dataclass(frozen=True)
class MacOpticalDrive:
    bsd_name: str        # e.g. "disk4"
    device_path: str     # "/dev/disk4"
    raw_path: str        # "/dev/rdisk4"
    vendor: str
    product: str
    is_ejectable: bool


def list_optical_drives() -> list[MacOpticalDrive]:
    """Return all currently-attached optical drives (empty list on non-darwin)."""
    if sys.platform != "darwin":
        return []
    return _enumerate_via_iokit()


def has_audio_disc(drive: MacOpticalDrive) -> bool:
    """True if the drive contains an Audio CD right now."""
    # macOS mounts audio CDs as a pseudo-filesystem under /Volumes/Audio CD.
    # The most reliable signal is `drutil status -drive <bsd>`; we shell to it.
    if sys.platform != "darwin":
        return False
    return _drutil_has_audio_cd(drive.bsd_name)
```

```
def eject_drive(drive: MacOpticalDrive) -> bool:
    if sys.platform != "darwin":
        return False
    try:
        subprocess.run(["/usr/bin/drutil", "tray", "eject", drive.bsd_name],
                       check=True, timeout=15)
        return True
    except (subprocess.CalledProcessError, subprocess.TimeoutExpired) as exc:
        LOG.warning("drutil eject failed for %s: %s", drive.bsd_name, exc)
        return False
```

The two private helpers:

```
def _enumerate_via_iokit() -> list[MacOpticalDrive]:
    """Use IOKit to find every IOCDBlockStorageDevice / IODVDBlockStorageDevice."""
    from Foundation import NSString  # noqa: F401  (forces PyObjC bootstrap)
    import objc
    iokit = objc.loadBundle(
        "IOKit", globals(),
        bundle_path=objc.pathForFramework("/System/Library/Frameworks/IOKit.framework"),
    )
    # Use IOServiceMatching("IOCDBlockStorageDevice") and IOServiceGetMatchingServices.
    # For each match: query "BSD Name", "Vendor Name", "Product Name", "Ejectable".
    # Return a list of MacOpticalDrive.
    ...


def _drutil_has_audio_cd(bsd_name: str) -> bool:
    try:
        out = subprocess.check_output(
            ["/usr/bin/drutil", "status", "-drive", bsd_name],
            text=True, timeout=10,
        )
    except (subprocess.CalledProcessError, subprocess.TimeoutExpired, FileNotFoundError):
        return False
    # `drutil status` prints "Type: CD-DA" for audio CDs.
    return "CD-DA" in out or "Audio CD" in out
```

> `drutil` is a built-in CLI in macOS (`/usr/bin/drutil`) and is reliable across Big Sur–Sequoia. It is sandbox-safe because we call it via `subprocess`, not by linking against any private framework.

3.2 Hot-plug events: `lyon/core/disc_watcher.py`

A long-lived watcher that emits Qt signals when a drive appears, disappears, or media is inserted/ejected. This is what lets the UI tabs come and go.

```
"""Live optical-drive notifications for macOS via DiskArbitration."""
from __future__ import annotations

import logging
import sys
from typing import Optional

from PySide6.QtCore import QObject, Signal, QTimer

LOG = logging.getLogger(__name__)
```

```
class OpticalDriveWatcher(QObject):
    drives_changed = Signal()          # any drive added/removed
    media_changed  = Signal(str)       # bsd_name of the drive whose media changed

    def __init__(self, parent: Optional[QObject] = None) -> None:
        super().__init__(parent)
        self._session = None  # DASession when on darwin
        self._poll_timer = QTimer(self)
        self._poll_timer.setInterval(2000)
        self._poll_timer.timeout.connect(self._poll_via_drutil)

    def start(self) -> None:
        if sys.platform != "darwin":
            return
        if self._start_disk_arbitration():
            return
        # Fallback: poll drutil every 2s. Cheap and reliable.
        self._poll_timer.start()

    def stop(self) -> None:
        self._poll_timer.stop()
        if self._session is not None:
            self._teardown_disk_arbitration()

    def _start_disk_arbitration(self) -> bool:
        try:
            import objc
            from CoreFoundation import CFRunLoopGetMain
            da = objc.loadBundle(
                "DiskArbitration", globals(),
                bundle_path=objc.pathForFramework(
                    "/System/Library/Frameworks/DiskArbitration.framework"
                ),
            )
            # DASessionCreate, DARegisterDiskAppearedCallback,
            # DARegisterDiskDisappearedCallback,
            # DARegisterDiskDescriptionChangedCallback,
            # DASessionScheduleWithRunLoop.
            # On any callback: self.drives_changed.emit() (or media_changed.emit).
            ...
            return True
        except Exception as exc:
            LOG.warning("DiskArbitration unavailable, falling back to drutil polling: %s", exc)
            return False

    def _poll_via_drutil(self) -> None:
        # Compare snapshots; emit drives_changed/media_changed accordingly.
        ...
```

Why offer both: DiskArbitration is the right API but requires more code and is harder to test headlessly in CI; the `drutil` polling fallback is two-second latency, zero-dependency, and works under any sandbox. Ship DA but keep the poller as the safety net.

3.3 Wire the watcher into `MainWindow`

File: `lyon/ui/main_window.py:115-190` (`MainWindow.__init__`)

Add — gated on darwin — instantiation of the watcher, connection of signals to a `_refresh_disc_tabs()` slot that calls `tab_bar.removeTab(...)` / `tab_bar.insertTab(...)` based on `disc_macos.list_optical_drives()`.

Storage of initial tab order needs adjustment so we can re-insert at the right index. Replace the constant `_TAB_ORDER` with computed visibility:

```
_ALL_TABS = ("Library", "Now Playing", "Podcasts", "Radio", "Video", "Disc", "Rip", "YouTube")
_DISC_TABS = {"Disc", "Rip"}

def _visible_tabs(self) -> tuple[str, ...]:
    if sys.platform == "win32":
        return self._ALL_TABS                       # Windows always shows everything
    if sys.platform == "darwin":
        if self._optical_drives_present():
            return self._ALL_TABS
        return tuple(t for t in self._ALL_TABS if t not in self._DISC_TABS)
    return tuple(t for t in self._ALL_TABS if t not in self._DISC_TABS)  # Linux dev
```

Whenever `drives_changed` fires, recompute and reconcile.

3.4 libdiscid disc ID lookup on macOS

`lyon/core/cd_detect.py` currently bails out at line 40 (`if sys.platform != "win32": return`). After Phase 2.5 preloads the bundled `libdiscid.0.dylib`, the Python `discid` package will work on macOS. Rewrite the platform guards to delegate to `disc_macos`:

```
def list_cd_drives() -> list[str]:
    if sys.platform == "win32":
        return _list_cd_drives_windows()
    if sys.platform == "darwin":
        from . import disc_macos
        return [d.device_path for d in disc_macos.list_optical_drives()]
    return []


def read_disc_toc(drive: str):
    if sys.platform == "win32":
        return _read_disc_toc_windows(drive)
    if sys.platform == "darwin":
        # libdiscid takes a device path like "/dev/disk4" directly.
        return _read_disc_toc_via_discid(drive)
    return None
```

`_read_disc_toc_via_discid(drive)` uses the existing `discid.read(device=drive)` call already wired into the Windows path.

3.5 Acceptance for Phase 3

- Plug in a USB SuperDrive with no disc — `list_optical_drives()` returns one entry, `has_audio_disc` returns False, Disc tab appears in the UI within 2 seconds.
- Insert an audio CD — `has_audio_disc` returns True; the Disc view populates with the track listing fetched via MusicBrainz.
- Eject the disc — the Disc view returns to its "no disc" placeholder.

- Unplug the drive — both Disc and Rip tabs vanish within 2 seconds. The user's currently-viewed tab does not change unexpectedly (if they were on Disc/Rip, fall back to Library).
- Re-plug the drive — tabs reappear.

Phase 4 — macOS Disc Ripping & Playback

Goal: The Rip view actually rips an audio CD into FLAC/MP3/etc. on macOS, and the Disc view can play tracks.

4.1 Ripping path: ffmpeg with libcdio

The bundled ffmpeg (from evermeet.cx static build) ships with `libcdio` support. The existing ripper code at `lyon/core/ripper.py:738-744` already checks for libcdio:

```
self._ffmpeg_has_libcdio = _ffmpeg_supports_demuxer(ff, "libcdio")
if self._ffmpeg_has_libcdio:
    self.log.emit("Using ffmpeg libcdio for CD audio input.")
```

The libcdio demuxer accepts input like `cdda:/dev/disk4:1` (track 1 from /dev/disk4). What you need to add:

- A small helper that maps a `MacOpticalDrive` + track index to the right input URL.
- In `ripper.py`'s read loop, when `sys.platform == "darwin"`, build the input URL via that helper instead of the Windows raw reader.

The Windows raw reader fallback (`_WindowsCddaReader`, `ripper.py:389-517`) stays — it only triggers on Windows when ffmpeg lacks libcdio. **No macOS counterpart is needed** because evermeet.cx ffmpeg always includes libcdio. Add a runtime assertion at startup:

```
if sys.platform == "darwin":
    ff = find_ffmpeg_binary()
    if ff and not _ffmpeg_supports_demuxer(ff, "libcdio"):
        LOG.error("Bundled ffmpeg lacks libcdio — disc rip will fail. Rebuild the bundle.")
```

4.2 Playback path: VLC CDDA MRL

File: `lyon/core/disc_playback.py:37-77`

Replace the Windows-only `vlc_device` helper with a platform branch:

```
def vlc_device_mrl(device_path: str) -> str:
    """Return a VLC cdda:// MRL for the given device path.

    Windows:  vlc_device("D:")        -> "cdda:///D:"
    macOS:    vlc_device("/dev/disk4") -> "cdda:///dev/disk4"
    """
    if sys.platform == "win32":
        return _vlc_device_windows(device_path)
    if sys.platform == "darwin":
        return f"cdda://{device_path}"
    raise NotImplementedError(f"CDDA MRL not supported on {sys.platform}")
```

Audit `lyon/ui/disc_view.py` for any place that displays a Windows drive letter directly to the user — on macOS prefer the friendly vendor+product strings collected in `MacOpticalDrive.product`.

4.3 CUETools Database verification

`lyon/core/ctdb_verify.py` already uses `subprocess` with `CREATE_NO_WINDOW` gated on Win32 (line 39). Verify it works on macOS by:

1. Confirming the ffmpeg-based CRC computation runs cross-platform.
2. Testing one full rip on macOS and seeing the verification result populate the Rip view's results column.

No code change expected unless paths leak through; fix on a case-by-case basis.

4.4 Eject after rip

`Settings.eject_after_rip` (default True). On macOS, route the eject through `disc_macos.eject_drive(drive)`. In `lyon/core/ripper.py:919-921`, replace the Windows-only `winmm.mciSendStringW` call with a platform dispatch:

```
if sys.platform != "win32":
    if sys.platform == "darwin":
        from . import disc_macos
        disc_macos.eject_drive(...)
    return
```

4.5 Acceptance for Phase 4

- Insert an audio CD; click "Rip"; choose FLAC; rip completes; output files are valid FLAC and play in another player.
- AccurateRip / CTDB verification column shows a result (pass/fail/not-in-db).
- Eject-after-rip ejects the disc.
- Click a track in Disc view → audio plays through libVLC.

Phase 5 — UI Parity

Goal: The Mac build looks like a Mac app — system font, native shortcut display, dynamic tab visibility, dark theme that respects platform conventions.

5.1 Stylesheet font fix

File: `lyon/ui/styles.py:13`

Replace the universal-selector font with a fallback stack and **drop the universal** `font-size`. Let `QApplication.setFont` (Phase 1.2) own the size.

```
* { color: #eef7ff; font-family: "Segoe UI", -apple-system, "SF Pro Text", "Helvetica Neue", "T
ahoma", sans-serif; }
```

Audit the rest of `styles.py` for `font-size` declarations that would clash with the macOS default 13pt: lines 41, 76, 83, 276–280, 344–350, 410–438, 442–465, 488–502, 559, 581, 591. The visible labels (transport title, lyrics, dialog title) should keep their explicit sizes; the smaller per-widget sizes (`8pt`, `font-size: 11px`) should be audited on Retina.

5.2 Shortcut labels

File: `lyon/ui/main_window.py:104-113` and the tooltip strings that bake "Ctrl" into user-visible text (lines 215, and ~40 occurrences inside `lyon/ui/knowledge_base_dialog.py`).

Create a tiny helper in `lyon/ui/widgets.py`:

```
from PySide6.QtGui import QKeySequence

def display_shortcut(seq: str) -> str:
    """Return a user-facing string for *seq*. On macOS, 'Ctrl+1' → '⌘1'."""
    return QKeySequence(seq).toString(QKeySequence.NativeText)
```

Use it everywhere a shortcut is displayed. The internal binding ("Ctrl+1") stays unchanged because Qt already maps `Ctrl` → ⌘ when constructing the actual `QKeySequence` on macOS.

For `knowledge_base_dialog.py`, replace each literal `<kbd>Ctrl+X</kbd>` with `<kbd>{display_shortcut("Ctrl+X")}</kbd>` at HTML-generation time.

5.3 Menu bar

`QMainWindow.menuBar()` automatically migrates to the screen menu bar on macOS — no code change needed at `main_window.py:375-426`. Verify after Phase 5.1 that the menu items render correctly.

One adjustment: Apple's HIG asks for a "Sea Lyon Media Manager" application menu with About / Preferences / Quit. Qt auto-creates it on macOS, and "About …" / "Preferences …" / "Quit" entries with magic role names (`QAction::AboutRole`, `QAction::PreferencesRole`, `QAction::QuitRole`) are moved automatically. Audit `lyon/ui/main_window.py` Help and Settings menus to set those roles:

```
about_action.setMenuRole(QAction.AboutRole)
settings_action.setMenuRole(QAction.PreferencesRole)
```

5.4 Dynamic tab gating (driven by Phase 3)

Already specified in 3.3. Acceptance criterion: with no drive plugged in, the tab bar shows 6 tabs (Library, Now Playing, Podcasts, Radio, Video, YouTube); plug in a drive, it grows to 8.

5.5 Acceptance for Phase 5

- All visible text uses San Francisco on macOS.
- Tab tooltips and Knowledge Base content show ⌘ where they previously said Ctrl on Mac; show Ctrl on Windows.
- The application menu (top-left of screen) shows About, Preferences, Quit under "Sea Lyon Media Manager".
- Tab list reacts to drive plug/unplug.

Phase 6 — Build Pipeline

Goal: A reproducible script (and matching CI job) that produces a signed, notarized `Sea Lyon Media Manager.app`, packaged inside `SeaLyonMediaManager-<ver>-arm64.dmg`, with every binary inside the bundle.

6.1 Generate `.icns` icon

`docs/brand/lyon-app-icon.png` is the source. Build script: `scripts/build-macos-icon.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
SRC="docs/brand/lyon-app-icon.png"
OUT="build/lyon-app-icon.iconset"
ICNS="build/lyon-app-icon.icns"
rm -rf "$OUT"; mkdir -p "$OUT"
for size in 16 32 64 128 256 512; do
  sips -z "$size" "$size"      "$SRC" --out "$OUT/icon_${size}x${size}.png"
  sips -z $((size*2)) $((size*2)) "$SRC" --out "$OUT/icon_${size}x${size}@2x.png"
done
iconutil -c icns "$OUT" -o "$ICNS"
```

6.2 PyInstaller spec — add darwin `BUNDLE`

File: `build/lyon.spec`

After the existing `COLLECT(...)` (line 141-150), append:

```
if sys.platform == "darwin":
    from lyon import __version__
    app_bundle = BUNDLE(
        coll,
        name="Sea Lyon Media Manager.app",
        icon=str(ROOT / "build" / "lyon-app-icon.icns"),
        bundle_identifier="com.drshoctopus.sealyonmediamanager",
        version=__version__,
        info_plist={
            "CFBundleName": "Sea Lyon Media Manager",
            "CFBundleDisplayName": "Sea Lyon Media Manager",
            "CFBundleShortVersionString": __version__,
            "CFBundleVersion": __version__,
            "CFBundleIdentifier": "com.drshoctopus.sealyonmediamanager",
            "LSMinimumSystemVersion": "11.0",
            "NSHighResolutionCapable": True,
            "NSHumanReadableCopyright": "© 2026 DrShoctopus",
            "LSApplicationCategoryType": "public.app-category.music",
            "NSLocalNetworkUsageDescription":
                "Sea Lyon discovers Chromecast and DLNA renderers on your local network.",
            "NSAppleEventsUsageDescription":
                "Used for system-wide media key handling.",
            "NSRemovableVolumesUsageDescription":
                "Sea Lyon accesses optical drives to detect and rip audio CDs.",
        },
    )
```

The PyInstaller `BUNDLE` step is the standard mechanism for producing a `.app` directory tree. It does not bundle libVLC by itself — that's the next step.

6.3 Post-build script: inject libVLC, plugins, libdiscid, fix dylib paths

After PyInstaller finishes, run `scripts/build-macos-bundle.sh`:

```
#!/usr/bin/env bash
set -euo pipefail

APP="dist/Sea Lyon Media Manager.app"
FW="$APP/Contents/Frameworks"
BIN="$APP/Contents/MacOS/bin"
VLC_STAGE="$(mktemp -d)/VLC.app"

mkdir -p "$FW/plugins" "$BIN"

# Mount the official VLC dmg (downloaded earlier in CI) and copy what we need.
hdiutil attach "$STAGED_VLC_DMG" -mountpoint /Volumes/SeaLyonVLC -nobrowse
cp -R "/Volumes/SeaLyonVLC/VLC.app" "$VLC_STAGE"
hdiutil detach /Volumes/SeaLyonVLC

cp "$VLC_STAGE/Contents/MacOS/lib/libvlc.dylib"     "$FW/"
cp "$VLC_STAGE/Contents/MacOS/lib/libvlccore.dylib" "$FW/"
cp -R "$VLC_STAGE/Contents/MacOS/plugins/." "$FW/plugins/"

# libdiscid (downloaded separately, see CI workflow).
cp "$STAGED_LIBDISCID_DYLIB" "$FW/libdiscid.0.dylib"

# ffmpeg + fpcalc — static arm64 binaries downloaded by CI.
cp "$STAGED_FFMPEG" "$BIN/ffmpeg"; chmod +x "$BIN/ffmpeg"
cp "$STAGED_FPCALC" "$BIN/fpcalc"; chmod +x "$BIN/fpcalc"

# Fix dylib install names so they resolve via @rpath inside the bundle.
install_name_tool -id @rpath/libvlc.dylib     "$FW/libvlc.dylib"
install_name_tool -id @rpath/libvlccore.dylib "$FW/libvlccore.dylib"
install_name_tool -id @rpath/libdiscid.0.dylib "$FW/libdiscid.0.dylib"
install_name_tool -change \
    "@loader_path/libvlccore.dylib" "@rpath/libvlccore.dylib" \
    "$FW/libvlc.dylib"

# Every VLC plugin links against libvlccore — rewrite each one.
for plugin in "$FW/plugins"/**/*.dylib; do
    install_name_tool -change \
        "@loader_path/../../libvlccore.dylib" "@rpath/libvlccore.dylib" \
        "$plugin" 2>/dev/null || true
done

# ------------------------------------------------------------------------
# Thin every universal binary inside the bundle down to arm64-only.
# PyObjC and PySide6 wheels are universal2, so PyInstaller drags in dylibs and
# .so extensions that contain x86_64 slices. We strip those slices here so the
# .app physically cannot launch on Intel Macs (not even under Rosetta).
# ------------------------------------------------------------------------
echo "Thinning universal binaries to arm64…"
find "$APP" -type f \( -name "*.dylib" -o -name "*.so" \) -print0 |
while IFS= read -r -d '' f; do
    archs=$(lipo -archs "$f" 2>/dev/null || true)
    case "$archs" in
```

```
        "arm64") ;;                             # already arm64-only, skip
        *arm64*)                                # universal -> thin to arm64
            tmp="${f}.arm64.tmp"
            lipo -thin arm64 "$f" -output "$tmp"
            mv "$tmp" "$f"
            ;;
        "")  ;;                                 # not a Mach-O (e.g. Python .so on Linux); skip
        *)
            echo "FAIL: $f is '$archs' with no arm64 slice"; exit 1 ;;
    esac
done

# Also thin the main app executable and any helper executables.
for f in "$APP/Contents/MacOS/LyonMusicManager" "$BIN/ffmpeg" "$BIN/fpcalc"; do
    archs=$(lipo -archs "$f" 2>/dev/null || true)
    case "$archs" in
        "arm64") ;;
        *arm64*) tmp="${f}.arm64.tmp"; lipo -thin arm64 "$f" -output "$tmp"; mv "$tmp" "$f" ;;
        *) echo "FAIL: $f arch '$archs' (expected arm64)"; exit 1 ;;
    esac
done

# Hard verification: NOTHING in the bundle may contain an x86_64 slice.
echo "Verifying bundle is 100% arm64…"
violations=0
while IFS= read -r -d '' f; do
    if file "$f" 2>/dev/null | grep -q "Mach-O"; then
        archs=$(lipo -archs "$f" 2>/dev/null || echo "")
        if [ "$archs" != "arm64" ]; then
            echo "FAIL ($archs): $f"
            violations=$((violations + 1))
        fi
    fi
done < <(find "$APP" -type f -print0)
if [ "$violations" -gt 0 ]; then
    echo "Bundle contains $violations non-arm64 binaries — refusing to ship."
    exit 1
fi
echo "Bundle staging complete: every Mach-O is arm64-only."
```

`install_name_tool` rewriting is the part that most often goes wrong. After this step, run `otool -L "$FW/libvlc.dylib"` and confirm the only paths shown are `@rpath/*`, `/usr/lib/*`, and `/System/Library/*`. Any absolute path to `/opt/homebrew/*` or the build server's filesystem must be rewritten.

Why this matters: without the thinning loop, the `.app` would contain ~200 universal dylibs from PyObjC, PySide6, and CPython itself. Each carries a full x86_64 slice that would let macOS launch the app under Rosetta on Intel hardware, against our explicit "100% Apple Silicon" guarantee. The `file` + `lipo -archs` check above is the gate that makes Intel launch physically impossible.

6.4 Entitlements file: `build/lyon.entitlements`

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.
0.dtd">
<plist version="1.0">
<dict>
```

```
    <key>com.apple.security.cs.allow-unsigned-executable-memory</key>
    <true/>
    <key>com.apple.security.cs.disable-library-validation</key>
    <true/>
    <key>com.apple.security.cs.allow-dyld-environment-variables</key>
    <true/>
    <key>com.apple.security.network.client</key>
    <true/>
    <key>com.apple.security.network.server</key>
    <true/>
    <key>com.apple.security.device.audio-input</key>
    <false/>
    <key>com.apple.security.files.user-selected.read-write</key>
    <true/>
</dict>
</plist>
```

`disable-library-validation` is required because we ship third-party dylibs (libVLC, libdiscid) signed by us, not by VideoLAN. `allow-dyld-environment-variables` is needed because the libVLC discovery code sets `DYLD_FALLBACK_LIBRARY_PATH`. `allow-unsigned-executable-memory` is needed by libVLC's plugin loader.

6.5 Codesign

```
DEV_ID="Developer ID Application: DrShoctopus (TEAM_ID_HERE)"

# Sign every bundled dylib and binary first (deep-first order).
find "$APP/Contents" \( -name "*.dylib" -o -path "*/bin/*" \) -type f -exec \
    codesign --force --sign "$DEV_ID" \
        --entitlements build/lyon.entitlements \
        --options runtime --timestamp {} \;

# Sign the main app bundle last.
codesign --force --sign "$DEV_ID" \
    --entitlements build/lyon.entitlements \
    --options runtime --timestamp \
    --deep "$APP"

codesign --verify --deep --strict --verbose=2 "$APP"
spctl --assess --type execute --verbose "$APP"
```

6.6 Notarize

```
ditto -c -k --keepParent "$APP" "dist/upload.zip"
xcrun notarytool submit "dist/upload.zip" \
    --apple-id "$APPLE_ID" \
    --team-id "$APPLE_TEAM_ID" \
    --password "$APPLE_APP_PASSWORD" \
    --wait
xcrun stapler staple "$APP"
xcrun stapler validate "$APP"
```

A typical notarization completes in 5-15 minutes. The `--wait` flag blocks the build step until Apple returns success/failure.

6.7 Package the DMG

```
VERSION=$(python -c "from lyon import __version__; print(__version__)")
DMG_OUT="dist/SeaLyonMediaManager-${VERSION}-arm64.dmg"

create-dmg \
    --volname "Sea Lyon Media Manager" \
    --window-pos 200 120 \
    --window-size 600 380 \
    --icon-size 110 \
    --icon "Sea Lyon Media Manager.app" 170 190 \
    --hide-extension "Sea Lyon Media Manager.app" \
    --app-drop-link 430 190 \
    --background "docs/brand/dmg-background.png" \
    "$DMG_OUT" \
    "$APP"

codesign --force --sign "$DEV_ID" --timestamp "$DMG_OUT"
xcrun notarytool submit "$DMG_OUT" \
    --apple-id "$APPLE_ID" --team-id "$APPLE_TEAM_ID" \
    --password "$APPLE_APP_PASSWORD" --wait
xcrun stapler staple "$DMG_OUT"
```

`create-dmg` installs via Homebrew (`brew install create-dmg`) on the build host. If you'd rather avoid that dependency, swap for `hdiutil create -format UDZO -srcfolder dist/dmg-staging "$DMG_OUT"` with a pre-staged folder containing `Sea Lyon Media Manager.app` and a symlink to `/Applications`.

6.8 Acceptance for Phase 6

- The DMG is ~120-150 MB (smaller than universal2 because all x86_64 slices are stripped).
- Double-click DMG → drag .app to /Applications → launch from /Applications → no Gatekeeper warning.
- `codesign --verify --deep --strict "$APP"` exits 0.
- `spctl --assess --type execute "$APP"` reports "accepted source=Notarized Developer ID".
- **Architecture audit passes:** `find "$APP" -type f -exec sh -c 'file "$1" | grep -q Mach-O && lipo -archs "$1"' _ {} \; | sort -u` outputs exactly one line: `arm64`. No `x86_64`, no `Mach-O universal`.
- **Intel-launch sanity test:** the bundle, when copied to an Intel Mac, refuses to launch with "The application can't be opened" (this is the macOS guarantee when no x86_64 slice exists).
- All of Phase 1–5 acceptance criteria still hold for the installed copy from the DMG.

Phase 7 — CI Workflow

Goal: Tagging `v1.x.0` on `LMM-DEV` produces both the Windows installer (existing job) and the macOS DMG (new job), uploaded to the same GitHub Release.

7.1 New file: `.github/workflows/macos-build.yml`

Outline (full file written during implementation):

```
name: macOS build

on:
  workflow_dispatch:
  push:
    tags:
      - 'v*.*.*'

env:
  FFMPEG_URL: "https://evermeet.cx/ffmpeg/getrelease/zip"
  FPCALC_URL: "https://github.com/acoustid/chromaprint/releases/download/v1.5.1/chromaprint-fpc
alc-1.5.1-macos-arm64.tar.gz"
  VLC_VERSION: "3.0.21"
  LIBDISCID_VERSION: "0.6.4"
  PYTEST_FLOOR: "680"

jobs:
  build:
    runs-on: macos-14   # Apple Silicon
    permissions:
      contents: write

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with: { python-version: "3.11", architecture: "arm64", cache: "pip",
                cache-dependency-path: requirements-build.txt }

      - name: Confirm arm64 host + Python
        run: |
          hw=$(uname -m)
          py=$(python -c "import platform; print(platform.machine())")
          [ "$hw" = "arm64" ] || { echo "Host is $hw, expected arm64"; exit 1; }
          [ "$py" = "arm64" ] || { echo "Python is $py, expected arm64 (not Rosetta)"; exit 1;
}

      - name: Install Python deps
        run: python -m pip install -r requirements-build.txt

      - name: Run pytest gate
        env: { QT_QPA_PLATFORM: offscreen }
        run: |
          collect=$(python -m pytest --collect-only -q | tail -3)
          count=$(echo "$collect" | grep -oE '^[0-9]+ tests collected' | grep -oE '^[0-9]+')
          [ "$count" -ge "$PYTEST_FLOOR" ] || { echo "below floor"; exit 1; }
          python -m pytest -q

      - name: Cache native binaries
        uses: actions/cache@v4
        with:
          path: vendor-mac
          key: mac-vendor-vlc${{ env.VLC_VERSION }}-discid${{ env.LIBDISCID_VERSION }}

      - name: Fetch & verify ffmpeg / fpcalc / libVLC / libdiscid
        run: scripts/fetch-macos-vendor-binaries.sh

      - name: Build icon
        run: scripts/build-macos-icon.sh

      - name: Write Last.fm API secrets
```

```
      env: { LASTFM_KEY: ${{ secrets.LYON_LASTFM_API_KEY }},
             LASTFM_SECRET: ${{ secrets.LYON_LASTFM_API_SECRET }} }
      run: |
        printf 'LASTFM_API_KEY = "%s"\nLASTFM_API_SECRET = "%s"\n' \
          "$LASTFM_KEY" "$LASTFM_SECRET" > lyon/core/_secrets.py

    - name: PyInstaller (arm64-only)
      run: python -m PyInstaller --noconfirm --target-arch arm64 build/lyon.spec

    - name: Inject libVLC + plugins + ffmpeg + fpcalc + libdiscid
      env:
        STAGED_VLC_DMG: vendor-mac/vlc.dmg
        STAGED_LIBDISCID_DYLIB: vendor-mac/libdiscid.0.dylib
        STAGED_FFMPEG: vendor-mac/ffmpeg
        STAGED_FPCALC: vendor-mac/fpcalc
      run: scripts/build-macos-bundle.sh

    - name: Import signing certificate
      env: { CERT_P12: ${{ secrets.APPLE_DEVELOPER_ID_CERT_P12 }},
             CERT_PASSWORD: ${{ secrets.APPLE_DEVELOPER_ID_CERT_PASSWORD }} }
      run: scripts/import-codesign-cert.sh

    - name: Codesign
      env: { APPLE_TEAM_ID: ${{ secrets.APPLE_TEAM_ID }} }
      run: scripts/codesign-macos-bundle.sh

    - name: Notarize & staple .app
      env:
        APPLE_ID: ${{ secrets.APPLE_ID }}
        APPLE_TEAM_ID: ${{ secrets.APPLE_TEAM_ID }}
        APPLE_APP_PASSWORD: ${{ secrets.APPLE_APP_PASSWORD }}
      run: scripts/notarize-macos-app.sh

    - name: Build DMG
      run: |
        brew install create-dmg
        scripts/build-macos-dmg.sh

    - name: Compute SHA256SUMS
      run: |
        ver=$(python -c "from lyon import __version__; print(__version__)")
        shasum -a 256 "dist/SeaLyonMediaManager-${ver}-arm64.dmg" \
                    > "dist/SeaLyonMediaManager-${ver}-macos-SHA256SUMS.txt"

    - name: Upload artifact
      uses: actions/upload-artifact@v4
      with:
        name: SeaLyonMediaManager-${{ github.ref_name }}-macos
        path: |
          dist/*.dmg
          dist/*-macos-SHA256SUMS.txt

    - name: Attach to GitHub Release
      if: startsWith(github.ref, 'refs/tags/')
      uses: softprops/action-gh-release@v2
      with:
        draft: true
        files: |
          dist/*.dmg
          dist/*-macos-SHA256SUMS.txt
```

7.2 Scripts to add under `scripts/`

Script	Purpose
`fetch-macos-vendor-binaries.sh`	Download + hash-verify ffmpeg, fpcalc, libVLC dmg, libdiscid into `vendor-mac/`
`build-macos-icon.sh`	Generate `.icns` from PNG (shown in 6.1)
`build-macos-bundle.sh`	Inject native bins, fix dylib paths (shown in 6.3)
`import-codesign-cert.sh`	Import `.p12` into a transient keychain
`codesign-macos-bundle.sh`	Sign every binary + the bundle (shown in 6.5)
`notarize-macos-app.sh`	Zip, submit, wait, staple (shown in 6.6)
`build-macos-dmg.sh`	`create-dmg` invocation + post-sign + notarize the DMG (shown in 6.7)

Each script should be `set -euo pipefail`, idempotent, and runnable locally for dev iteration.

7.3 Update the Windows workflow

`.github/workflows/windows-build.yml` already creates a draft GitHub Release on tag push. The new macOS workflow attaches DMG artifacts to the same release. Coordinate the two jobs by:

- Both fire on `push: tags: ['v*.*.*']`.
- Both use `softprops/action-gh-release@v2` with `draft: true` and **append** to the existing release rather than recreating it.

7.4 Acceptance for Phase 7

- Push tag `v1.1.0` → Windows job and macOS job run in parallel.
- ~30 min later (notarization is the slow step), a draft GitHub Release contains: `…-Setup.exe`, `…-windows.zip`, `…-windows-SHA256SUMS.txt`, `…-arm64.dmg`, `…-macos-SHA256SUMS.txt`.
- Downloading the DMG on a personal M1/M2 Mac, dragging to Applications, and launching from /Applications produces no Gatekeeper warning.

Phase 8 — Updater (in parallel with Phase 6)

Goal: macOS users get update notifications pointing at the `.dmg`, not the `.exe`.

8.1 Appcast generator

File: `scripts/generate-appcast.py`

Add two enclosures per release (Windows and macOS), each carrying `sparkle:os` and `sparkle:minimumSystemVersion`:

```
<item>
  <title>Sea Lyon Media Manager 1.1.0</title>
  <pubDate>...</pubDate>
  <sparkle:version>1.1.0</sparkle:version>
```

```
  <enclosure
      url="https://github.com/.../SeaLyonMediaManager-1.1.0-Setup.exe"
      length="..." type="application/octet-stream"
      sparkle:os="windows"/>
  <enclosure
      url="https://github.com/.../SeaLyonMediaManager-1.1.0-arm64.dmg"
      length="..." type="application/octet-stream"
      sparkle:os="macos"
      sparkle:minimumSystemVersion="11.0"/>
</item>
```

CLI arg changes: `--macos-installer-url`, `--macos-installer-size`, plus the existing Windows ones.

8.2 Client-side filtering

File: `lyon/core/updater.py`

When parsing the feed, filter `<enclosure>` by `sparkle:os` (or by file extension as a fallback). On Windows pick the `.exe`; on macOS pick the `.dmg`.

8.3 Update dialog copy

File: `lyon/ui/update_dialog.py`

Add a one-line platform-aware string:

- Windows: "Download Installer"
- macOS: "Download Disk Image"

And in the body, "Run the installer to update." vs "Open the disk image and drag the app to Applications to update."

8.4 Acceptance for Phase 8

- macOS client polling the appcast sees the macOS enclosure and downloads the DMG.
- Windows client unchanged.

Phase 9 — QA, Sign-off, and Release

9.1 Manual test matrix (run before tagging)

Run the four-row matrix from Pre-flight 0.4 on real hardware. Each row should produce screenshots + a checklist signed by the tester. Don't ship without all four green.

9.2 Additional smoke tests

- Open library: scan ~10k tracks from an external drive. No regressions vs Windows on scan time.
- Stream a radio station for ≥ 5 minutes. ICY metadata updates.

- YouTube download an audio track at FLAC; verify file plays.
- Toggle equalizer at three different presets; audio changes audibly.
- DLNA: switch on the in-app DLNA server, open VLC on another machine, browse "Sea Lyon Media Manager", play a track. (Expect a macOS "Allow local network access?" prompt the first time.)
- Cast: discover and play to a Chromecast Audio. (Same local-network prompt.)
- Last.fm scrobble: scrobble one track, verify it appears on last.fm/user/…

9.3 Versioning

Bump `__version__` in `lyon/__init__.py` (e.g. `1.1.0`). Add a CHANGELOG section noting macOS Apple Silicon support. Tag `v1.1.0`. Push tag — both CI workflows fire.

9.4 Release notes

Add a `## macOS` section to the release notes describing:

- **Apple Silicon (arm64) only.** Requires an M1, M2, M3, M4 (or later) Mac. **Intel Macs are not supported, even via Rosetta** — the binaries contain no x86_64 slices and will refuse to launch on Intel hardware.
- Minimum macOS 11 (Big Sur).
- All binaries bundled — no Homebrew, no separate VLC.app install.
- Optical disc support is automatic; Disc/Rip tabs appear when a drive is attached.

Also update `README.md` "System Requirements" with the same arm64-only language so users self-select before downloading.

9.5 Post-release watch

For one week after release, watch for:

- Notarization failures reported by users (most often: Gatekeeper quarantine on the DMG itself).
- libVLC plugin load failures (a sign that `install_name_tool` rewriting missed a path).
- Crashes on M3 / Sequoia that don't reproduce on M1 / Big Sur.

Summary Checklist

☐ **Phase 0** — Apple Developer cert, GitHub secrets, test hardware booked. Arm64-only host confirmed.

☐ **Phase 1** — `app_data_dir`, font, window-mode, `requirements.in` deps.

☐ **Phase 2** — libVLC + ffmpeg + fpcalc + libdiscid runtime discovery, arm64-only sources verified.

☐ **Phase 3** — `disc_macos.py`, `disc_watcher.py`, IOKit/DiskArbitration, tab gating.

☐ **Phase 4** — ffmpeg+libcdio ripping; `cdda:///dev/disk*` playback; eject via drutil.

☐ **Phase 5** — QSS font fallback, NativeText shortcut display, menu roles, dynamic tabs.

☐ **Phase 6** — `.icns`, BUNDLE block, native-binary injection, `lipo` thinning to arm64, codesign, notarize, DMG.

☐ **Phase 6 arch gate** — every Mach-O in the bundle returns `arm64` from `lipo -archs`. Zero x86_64 slices anywhere.

☐ **Phase 7** — `macos-build.yml` workflow with `--target-arch arm64`, runner arch assertions, 7 build scripts.

☐ **Phase 8** — Per-OS appcast enclosures, client-side filter, dialog copy.

☐ **Phase 9** — QA matrix on real arm64 hardware, version bump, tag, ship. README + release notes call out Apple Silicon-only.

When every box is ticked, you have an Apple Silicon build that behaves like a native Mac app, contains zero x86_64 code, includes everything a user needs in a single signed DMG, supports CD/DVD drives when they're plugged in and hides the related UI when they aren't — without forking the Windows codebase.

End of plan.